Public

OPEN PLATFORM FIRMWARE (OPF)

Trusted, AI-Ready Open Firmware: From Auditable Foundations to Agentic Lifecycle Automation

Paul Grimes *Principal Member of Technical Staff, AMD*

Chitrak Gupta *Senior Director, AMI*

OCP PLATINUM Member

Public

Modularity Moved the Focus on to Firmware

DC-MHS and M-PnP changed how firmware needs to be built

- Compute, power, I/O and management are now independently sourced modules
- Every HPM, DC-SCM and riser combination is a distinct firmware target
- One module swap touches FRU and mPnP JSON, device tree, defconfig, Yocto layers, Redfish schema and BIOS DSC/FDF/PCD
- Bring-up serializes behind manual porting, so firmware sets the platform schedule

And security compliance is paramount

- SBOM and vulnerability reporting are shipping requirements under the Cyber Resilience Act and equivalent regimes
- CVEs must be tracked across upstream OpenBMC and EDK2 and every downstream fork
- Auditability is now a property of the build system, not a document written at the end

Modularity is only “plug-and-play” if firmware keeps up. Hence a secure firmware pipeline with Automation is the need.

Public

Four Automation Layers, Built on an AI-Ready Repository

4 — Agentic end-to-end orchestration

A chatbot and typed tools drive setup, build, test, attestation and deployment as one request.

3 — Security and compliance in the build

Vulnerability management and SBOM emitted as build outputs; AMILiA for analysis and triage.

2 — AI assistance for firmware development

Local models generate BMC and BIOS code and turn build, boot and test logs into root cause.

1 — Build orchestration for firmware

Declarative platform manifest, pinned sources, hermetic containerized builds, reproducible digests.

0 — AI-ready repository foundation

Versioned context, scoped rules, playbooks, guardrails, and evidence.



Each layer is only trustworthy if the one below it is. We build Security across layers.

Public

Rung 0 — Make the Repository AI-Ready

- AI-generated changes reflect the source code plus the instructions and context guiding the assistant
- Today, that guidance often lives in private files and personal settings, so results vary and reviewers cannot see the full basis for a change
- An AI-ready repository makes architecture, boundaries, conventions, build and test expectations, and constraints shared, version-controlled project context
- Every configured assistant begins from the same project baseline
- Because the context lives in the repository, it can be reviewed, improved, and reused across contributors and tools

Before an agent can operate safely and effectively, it needs grounded project context.

Public

Rung 0 — The AI-Ready Repository Stack

- Project context: Repository maintainers use an AGENTS.md to define project purpose, structure, ownership, and approved build and test commands
- Scoped rules: File patterns select the right guidance—for example, C rules for C files, build rules for build files.
- Playbooks: Maintainers define the inputs, ordered steps, required evidence, and output format that assistants follow for recurring tasks
- Guardrails: Always-on instructions define required and prohibited behavior; hooks can enforce high-risk checks before actions run
- Evidence access: MCP and API integrations let assistants retrieve current issues, pull requests, CI logs, specifications, and design documents directly from their source systems.
- Tool portability: The project maintains one shared set of instructions, providing the same guidance to agents in each AI tool's supported format.

The repository—not the model—is the durable trust layer.

Public

Rung 0 — openSIL POC Example: Shared Context in Practice

- The openSIL root AGENTS.md records its architecture, repository boundaries, Meson build and test commands, and critical constraints
- File-matching instruction files apply C, Meson, and silicon-IP guidance to the relevant source and build files
- Engineers invoke PR-review and CI-triage playbooks; assistants analyze actual diffs and logs and return consistently structured findings
- Repository guardrails require assistants to preserve safe Git workflows, protect secrets, run required checks, and leave approval to engineers
- When a recurring problem is found, an engineer or agent proposes an instruction or playbook update for human review
- Once the update is merged, every configured assistant uses the revised guidance in future work

We plan to propose this shared foundation as a reusable OPF contribution.

Public

Rung 1 — Deterministic Firmware Build Orchestration

What the firmware management pipeline must guarantee

- Declarative platform manifest: module identity maps to source layers, defconfig, device tree and firmware-volume layout
- Pinned upstream revisions and containerized toolchains, so Yocto and EDK2 builds are hermetic
- One trigger, multiple configuration: every HPM variant built into a single addressable artifact namespace
- Deterministic output: the same manifest and revisions produce the same image digest

Why nothing above this rung works without it

- AI output is only reviewable when the build is reproducible and the results are deterministic
- An agent needs a stable, typed action surface: build, flash, test, scan, report
- Provenance starts here. We cannot attest what you cannot reproduce
- Clean builds make failures signal rather than noise, which is what makes log intelligence possible

Orchestration first. Determinism is the contract that everything agentic plugs into.

Public

Rung 2 — AI Assisted Code Generation

OpenBMC core and APIs

- Generate and modify Redfish resource handlers from FRU and mPnP JSON plus I²C and schematic maps
- Device tree descriptors, kernel defconfig overlays, Yocto layer and recipe edits
- Sensor porting code derived from schematics instead of transcribed by hand
- Every change checked against the DMTF Redfish Interop Validator before it reaches review

BIOS and UEFI

- Parse module identity and system manifest, then edit DSC, FDF and PCD accordingly
- Add required drivers and libraries and place them correctly in firmware volumes
- Compute memory, power, thermal and GPIO platform settings
- Detect → Interpret → Configure: static at build time, bounded and pre-enumerated at boot

The model runs inside the build enclave: source, schematics and design documents never leave it. No third-party inference on proprietary or NDA-bound IP.

Public

Rung 2 — Turning Logs Into Root Cause

Build log

Error and warning clustering across the whole matrix

Fix recommendation pointing at the offending recipe or DSC line

Ticket opened automatically with the diagnostic bundle attached

Boot log

Driver dispatch order reconstructed from the boot trace

ASSERT and hang triage against the exact source revision

Code-level fix suggestion surfaced in the developer's IDE

Test log

Post-run validation through the AI Log Analyzer

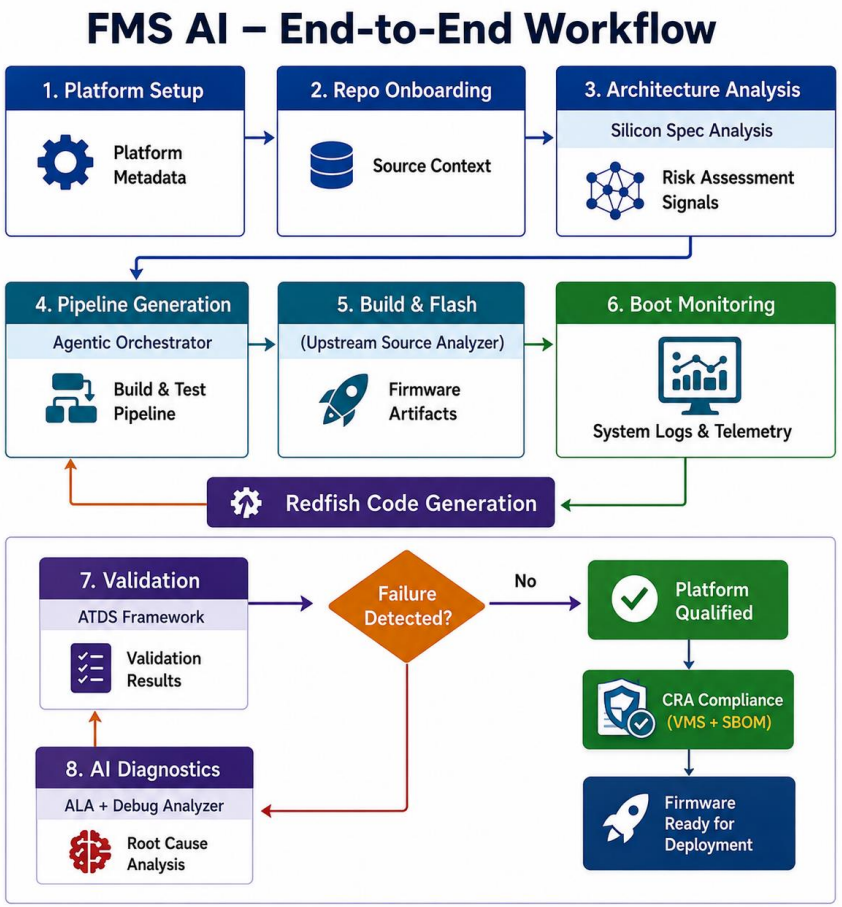
New failures separated from known regressions

Coverage gaps fed back into test generation

Delivered where engineers already are — a VS Code plugin, the CLI, and an automatic comment on the merge request. Thousands of log lines become a structured, navigable report.

Public

FMS AI: Rungs 1 and 2 as One Closed Loop



Reading the loop

Stages 1–3 — understand the platform: metadata, repository onboarding, and architecture and risk analysis over a knowledge graph.

Stages 4–6 — act: the agentic orchestrator emits a build and test pipeline, produces firmware artifacts, then watches the boot through system logs and telemetry.

Stages 7–8 — judge: validation decides the branch. A clean run goes to CRA compliance (VMS and SBOM) and on to deployment. A failure goes to AI diagnostics for root cause and re-enters at pipeline generation.

Redfish code generation sits across the middle because it is triggered by config change, not by stage. (BMC builds)

Nothing is promoted without passing back through validation and compliance. The loop has no shortcut.

Public

Rung 3 — Security as a Build Output

Vulnerability management (VMS)

- Continuous CVE and security-advisory correlation against every component in the image
- Applies equally to upstream OpenBMC and EDK2, to downstream forks, and to vendor drops
- Triage separates reachable from present-but-unreachable, with a fix or backport candidate
- Re-evaluated on every build, so an image's security posture is never stale

Software Bill of Materials

- SPDX and CycloneDX emitted as build artifacts, not written up afterwards
- Bound to the image digest, so the SBOM travels with the artifact it describes
- Diffed build over build: a new or changed component becomes a review event
- This is the record that CRA-style reporting obligations actually ask for

AI Chatbots can give this a conversational surface: ask which CVEs affect this image and why, across the SBOM, the advisory corpus and years of firmware knowledge — instead of reconciling spreadsheets.

Public

Keeping Source Repositories Safe in the Pipeline

1 Identity and access

SSO with short-lived, per-repository tokens. No long-lived CI credentials. Least privilege by default.

2 Integrity

Signed commits and signed tags, protected branches, and mandatory human review on every AI-authored change.

3 Upstream provenance

Upstream mirrored at pinned commits. Vendor and third-party drops scanned on ingest, never pulled live at build time.

4 Build isolation

Ephemeral containers, egress denied by default, source mounted read-only, secrets injected at point of use.

5 AI boundary

Self-hosted inference only. Source, schematics and design documents stay inside. Prompt and retrieved context logged for audit.

6 Attribution

Every generated artifact carries model name, version and prompt hash in the commit trailer, so any change traces to what produced it.

The repository is the source of truth, Agents get scoped, audited, revocable access to it

Public

Keeping Generated Artifacts Trustworthy

Sign and attest on the way out

- Image signing at build exit, with keys held in an HSM or KMS and never present on the builder
- Provenance attestation in the in-toto and SLSA style: which sources, which toolchain, which agent, which prompt
- Signature, SBOM and attestation stored together as one immutable bundle
- Content-addressed artifact store with retention sized for audit and regulatory reporting

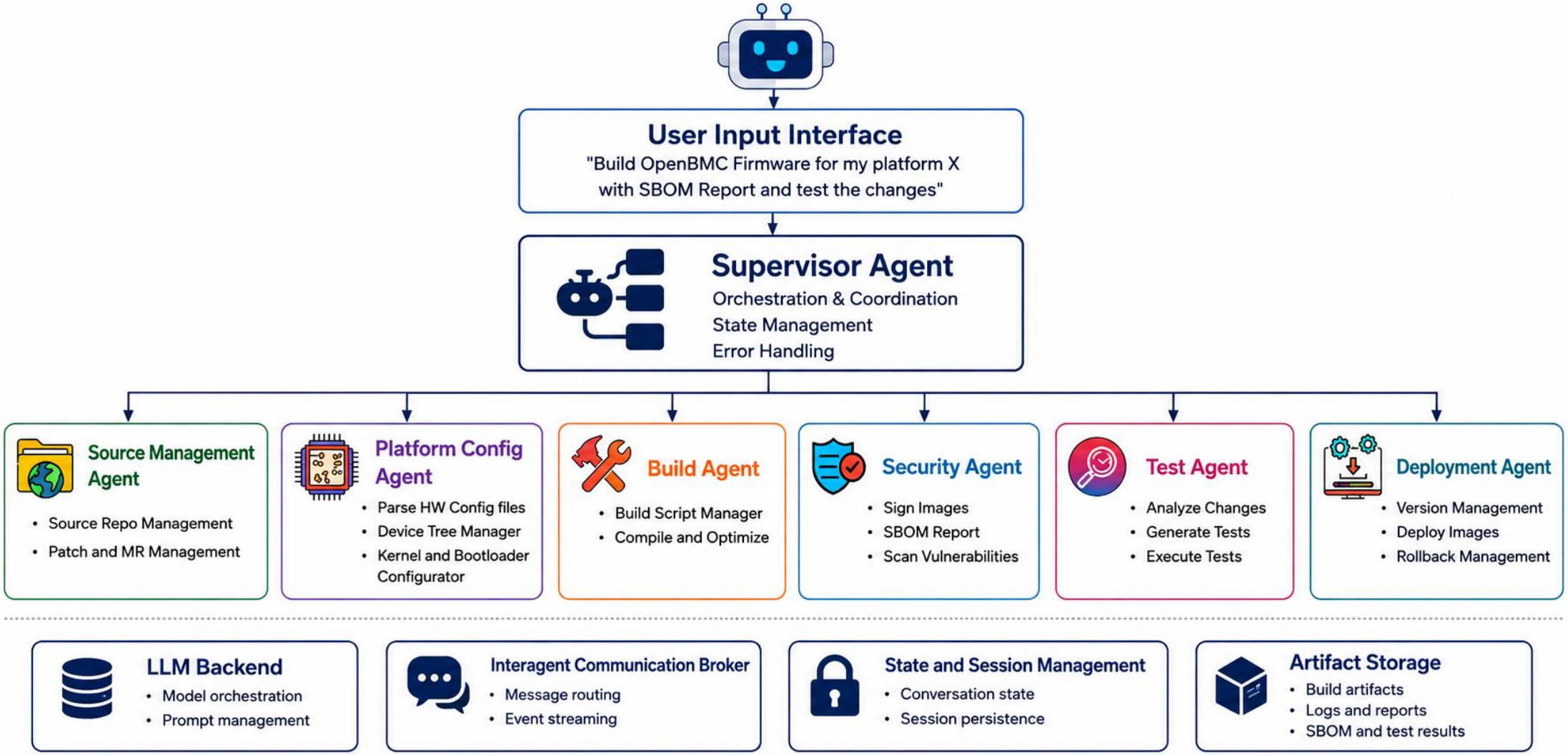
Verify and control on the way in

- Verified at flash time by the BMC and platform root of trust; measured boot extends the chain into runtime
- Promotion between development, qualification and release requires a passing attestation, not just a passing build
- Rollback restricted to a known-good signed image, with anti-rollback on revoked versions
- Revocation and re-attestation are pipeline operations, not manual exercises

The agent can build, test and stage. It cannot mint trust — signing, promotion and deployment stay behind human-approved gates.

Public

Rung 4 — One Request, Seven Agents



Public

Agents, Tools and Guardrails

Agents and the tools they may call

- Supervisor — orchestration, state management, error handling
- Source Management — repository access, patch and merge-request handling
- Platform Config — hardware config parsing, device tree, kernel and bootloader configuration
- Build — build script management, compile and optimise
- Security — image signing, SBOM generation, vulnerability scanning
- Test — change analysis, test generation, execution
- Deployment — version management, deploy, rollback
- Shared services: LLM backend, inter-agent broker, state and session store, artifact storage

What keeps it safe to run

- Every tool is a typed, allow-listed action with a schema. There is no general shell escape.
- Human-in-the-loop gates at merge, sign, promote and deploy — the agent proposes, a person approves
- Full session trace: prompt, retrieved context, tool call, artifact digest. Replayable for audit.
- Failure is a first-class path — bounded retries, then escalate with a diagnostic bundle rather than improvise
- Least privilege per agent: the Build agent cannot sign, the Security agent cannot push code

Public

What We Are Measuring

Platform porting time

Module swap to first qualified build

First-boot success rate

New HPM variant boots on the first generated image

Log to root cause

Failing log to actionable code-level finding

Test generation and coverage

Time to produce tests, and coverage of changed code

CVE triage decision time

Advisory published to reachable / not-reachable call

Attested build coverage

Share of builds shipping complete SBOM and provenance

Public

Call to Action

The problem to solve

- Modular OCP platforms is aiming for a platform for firmware build orchestration, provenance, and SBOM-to-image binding. Create an agentic pipeline to use the tools for expediting platform porting
- Move from Code to Specifications and Configurations and manage them in a secure way

How to get involved

- OCP Server, OpenRMC and Open Systems for AI workgroups, and the DC-MHS / M-PnP sub-projects
- AMI intends to contribute an open firmware management solution and supporting tools for mPnP FPGA and FRU json generation by Q1, 2027.
- Wanted: silicon and device vendors to co-develop the platform manifest schema, and reviewers for the provenance and attestation profile
- Come find us at the AMD and AMI Booths

Public

Thank You!

AMD

ami™

A LATTICE COMPANY



GENERAL NOTICE: Other entity and product names used in this publication are for identification purposes only and may be trademarks of their respective holders.